Plataforma de trabajo del curso

API REST, Docker y servicios de apoyo

IFCD107 – Especialista en Inteligencia Artificial

Octubre de 2026

Resumen

Este documento describe el entorno práctico del proyecto de regresión polinómica. La aplicación Flask expone una API HTTP para ejecutar modelos de aprendizaje automático; Docker empaqueta la aplicación y permite iniciar servicios auxiliares como Ollama, n8n y Flowise. Se explican los componentes, el intercambio de datos, la puesta en marcha, las pruebas y la gestión cotidiana del entorno.

1. Objetivo de la plataforma

El propósito es practicar dos habilidades complementarias: consumir y diseñar servicios mediante una API, y ejecutar aplicaciones en un entorno reproducible mediante contenedores. En vez de llamar directamente a las funciones Python desde cada herramienta, el cliente envía una petición HTTP a Flask. La API valida la entrada, ejecuta el algoritmo y devuelve una respuesta JSON con métricas, identificador de ejecución y, cuando aplica, rutas de los artefactos generados.

Docker permite preparar los componentes con versiones y configuración declaradas. Cada servicio se ejecuta en su contenedor, con sus dependencias aisladas; Docker Compose coordina su arranque, las redes internas, los puertos y los volúmenes persistentes.

2. Componentes del proyecto

Componente	Función en las prácticas
API Flask (`app.py`)	Publica rutas HTTP para salud, autenticación y entrenamientos. Delega la lógica de modelos en `api_functions.py` y `regression/`.
Dockerfile	Construye la imagen de la API sobre Python 3.11, instala `requirements.txt` y ejecuta Gunicorn en el puerto 5000.
Docker Compose	Declara los servicios y su configuración. El Compose raíz incluye la API, Ollama, n8n y Flowise, además del Compose oficial de Dify cuando su repositorio está preparado.
Ollama	Sirve modelos de lenguaje locales en la red de contenedores. La configuración descarga por defecto `llama3:8b` y `mistral:7b`.
n8n	Entorno visual para construir flujos de automatización que pueden llamar a APIs.
Flowise	Interfaz visual para crear flujos con modelos y componentes de IA.
Dify	Plataforma de aplicaciones LLM incorporada desde su Compose oficial; requiere preparar su repositorio y entorno.
Postman	Cliente para enviar solicitudes HTTP, adjuntar CSV y revisar respuestas sin escribir un cliente propio.

La API también se puede ejecutar directamente con Python, sin Docker. Esta opción es útil para desarrollar y depurar Flask; Docker resulta adecuado para practicar el despliegue de servicios relacionados. La colección de Postman está en `postman/Regresion_Polinomica.postman_collection.json` y los ejemplos de datos en `ejemplos/`.

3. Modelo mental: imagen, contenedor y servicio

Una **imagen** es la plantilla inmutable que contiene el programa y sus dependencias. Un **contenedor** es una instancia en ejecución de esa imagen. Un **servicio** de Compose describe cómo crear y ejecutar uno o más contenedores: imagen o construcción, variables, puertos, volúmenes y dependencias.

Los servicios de Compose se comunican entre sí por la red interna usando el nombre del servicio como dirección. Por ejemplo, la API usa `http://ollama:11434` para alcanzar Ollama. Desde el navegador del equipo, en cambio, se usa un puerto publicado como `http://localhost:5000`. El puerto interno y el publicado se conectan mediante una regla del Compose.

4. Recorrido de una petición a la API

1. Postman envía una solicitud HTTP a la dirección local de la API.
2. Flask identifica la ruta y el método, valida los datos y, para las rutas protegidas, comprueba el token Bearer.
3. La función correspondiente prepara los datos, separa entrenamiento y prueba, ajusta el modelo y calcula predicciones y métricas.
4. El proyecto guarda los resultados bajo `runs/<referencia>/`, incluyendo según el endpoint registros, CSV, gráficas o modelos.
5. Flask devuelve un estado HTTP y un cuerpo JSON. Postman muestra ambos para inspección.

Una API es un contrato entre cliente y servidor. El método indica la operación; la ruta identifica el recurso o acción; los encabezados transportan metadatos como el tipo de contenido y la autorización; el cuerpo contiene la entrada. JSON se utiliza para datos estructurados y `multipart/form-data` para formularios que incluyen ficheros.

5. Rutas representativas

Método	Ruta	Uso
GET	`/api/v1/health`	Comprobar que la API responde; no requiere token.
POST	`/api/v1/auth/token`	Enviar credenciales y obtener un token temporal.
GET	`/api/v1/regresion/parametros`	Consultar el catálogo de parámetros disponibles.
POST	`/api/v1/regresion/polinomica`	Ajustar el modelo a datos JSON.
POST	`/api/v1/regresion/polinomica/csv`	Ajustar el modelo a un CSV con columnas `x,y`.
POST	`/api/v1/regresion/polinomica/randomizedsearch`	Buscar hiperparámetros con validación cruzada.

POST	`/api/v1/random-forest/csv`	Entrenar Random Forest desde un CSV tabular.

Las rutas de entrenamiento requieren `Authorization: Bearer <token>`. La emisión del token y la comprobación de salud son públicas. El código de estado ayuda a reconocer el resultado: 2xx indica éxito, 400 entrada inválida, 401 falta de autorización, 413 exceso de tamaño y 5xx un fallo del servidor o del entrenamiento.

5.1. Ejemplo de solicitud JSON

```
POST http://localhost:5000/api/v1/regresion/polinomica
Authorization: Bearer <token>
Content-Type: application/json

{
  "reference": "prueba_curso",
  "degree": 2,
  "data": [
    {"x": 1, "y": 3.1},
    {"x": 2, "y": 5.2},
    {"x": 3, "y": 9.0},
    {"x": 4, "y": 15.1},
    {"x": 5, "y": 22.8}
  ]
}
```

Los campos exactos y límites se deben consultar en la guía del proyecto y en la colección Postman. Para CSV, Postman construye automáticamente el formulario multipart al adjuntar el archivo; no hay que fijar manualmente el encabezado boundary.

6. Puesta en marcha con Docker

El entorno descrito usa Docker Desktop con Docker Compose v2.20 o posterior. La primera ejecución descarga imágenes y modelos; Ollama puede necesitar varios gigabytes de almacenamiento, además de memoria y tiempo de CPU/GPU. Se necesita conexión a Internet para esas descargas iniciales.

El Compose raíz requiere los archivos de configuración locales. Si no existen, crear el entorno a partir de las plantillas; si ya existen, conservarlos y completar las variables que falten. El repositorio oficial de Dify y su archivo de entorno son necesarios si se va a levantar Dify:

```
if (!(Test-Path .env)) { Copy-Item .env.example .env }
git clone https://github.com/langgenius/dify.git dify
if (!(Test-Path dify/docker/.env)) {
  Copy-Item dify/docker/.env.example dify/docker/.env
}
```

Antes de iniciar, configurar credenciales y secretos aleatorios propios en los archivos `.env`; no publicar esos archivos ni copiar secretos a documentación compartida. Para Dify, la guía principal incluye la configuración de perfiles, sandbox y puertos. Si solo se quiere ejecutar la API localmente, se puede omitir este conjunto de servicios y usar el procedimiento Python de `README.md`.

Para construir e iniciar los servicios:

```
docker compose up -d --build
docker compose ps
docker compose logs -f regression-api
```

La opción `-d` deja los contenedores en segundo plano; `-build` reconstruye la imagen de la API. La descarga de modelos se realiza mediante el servicio de inicialización de Ollama y puede prolongar el primer arranque. Los servicios principales se publican localmente en:

Servicio	Dirección
API de regresión	http://localhost:5000
n8n	http://localhost:5678
Flowise	http://localhost:3001
Dify (si se configuró)	http://localhost:8080
Ollama	http://localhost:11434

Las interfaces locales están ligadas a la máquina anfitriona. Para comprobar la API, abrir http://localhost:5000/api/v1/health o ejecutar `Invoke-RestMethod http://localhost:5000/api/v1/health` en PowerShell.

7. Práctica guiada en Postman

1. Importar la colección de Postman del proyecto.
2. Configurar `baseUrl` como `http://localhost:5000`, el usuario `admin` y la contraseña tomada del entorno local autorizado.
3. Ejecutar primero **Obtener token**. La colección guarda el token para las siguientes peticiones.
4. Ejecutar **Health** y comprobar el estado HTTP y el JSON.
5. Probar el ajuste polinómico JSON y, después, el ajuste CSV con `ejemplos/datos_ejemplo_xy.csv`.
6. Inspeccionar métricas, `run_id` y archivos bajo `runs/`. Cambiar el grado o la referencia y repetir.
7. Probar una entrada inválida para observar cómo la API devuelve un error HTTP legible.

8. Datos, persistencia y limpieza

El servicio de la API monta un volumen Docker llamado `regression_runs` sobre `/app/runs`; así, los artefactos persisten aunque se reemplace el contenedor. Ollama guarda sus modelos en `./ollama`; n8n y Flowise tienen volúmenes propios para sus datos y configuración. Eliminar un contenedor no equivale a borrar el volumen.

Para detener el conjunto manteniendo los volúmenes:

```
docker compose down
```

Para iniciarlo otra vez, ejecutar `docker compose up -d`. Evitar `docker compose down -v` si se quieren conservar modelos, flujos, configuraciones y resultados. Para revisar un servicio concreto, usar `docker compose logs -tail 100 nombre-del-servicio`.

9. Buenas prácticas de trabajo

- Mantener claves, contraseñas y tokens en los archivos locales de entorno; compartir solo plantillas sin secretos.
- Usar referencias sencillas para agrupar ejecuciones y no incluir datos personales en ellas.
- Empezar con los CSV sintéticos de `ejemplos/`; están preparados para practicar y no representan registros reales.

- Verificar el estado del servicio y los logs antes de atribuir un error al algoritmo.
- Distinguir la dirección usada por el equipo (`localhost`) de la dirección usada entre contenedores (nombre de servicio, por ejemplo `ollama`).
- Recordar que un modelo local no envía la inferencia a un proveedor externo, pero sí utiliza recursos del equipo.

10. Resultados de aprendizaje

Al trabajar con esta plataforma, el alumnado practica la lectura de contratos HTTP, el uso de métodos y códigos de estado, el intercambio de JSON y CSV, la autenticación Bearer, el diagnóstico mediante logs y la ejecución de servicios con Docker Compose. En paralelo, puede observar cómo una API integra el entrenamiento de modelos y convierte sus resultados en respuestas y artefactos consultables.

11. Documentación del proyecto

Este texto sirve como presentación de la plataforma. Para detalles de implementación y contratos completos, consultar `documentacion/guia_proyecto.tex`; para instrucciones actualizadas de arranque, configuración y solución de problemas, consultar `README.md`. Las solicitudes reproducibles están en la colección de Postman.